ESPRIT • ÉCOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIE

Département Informatique • Module : Architecture des Systèmes d'Information

---

ÉTUDE DE CAS : AUTOLOC

Implémentation & Tests de la Couche Présentation (Controllers REST)

Étudiant : Sélim Hechmi

Dépôt GitHub : https://github.com/salim-hechmi/AutoLoc

Stack Technique : Spring Boot 4.1.1, Java 17/23, Spring Data JPA, Hibernate, MySQL 8.4, Postman

Année Universitaire : 2026 - 2027

1. Présentation du Travail Réalisé

Ce livrable présente la réalisation complète de la couche présentation (**Controllers REST**) pour le système de gestion de location de véhicules **AutoLoc**.

L'ensemble des objectifs du cahier des charges a été atteint :

- **9 Contrôleurs REST implémentés :** `AgenceController`, `ClientController`, `ContratController`, `EmployeController`, `EquipementController`, `MaintenanceController`, `PaiementController`, `ReservationController`, `VehiculeController`.
- **Annotations Spring appliquées :** `@RestController`, `@RequestMapping`, `@RequestBody`, `@PathVariable`, et `@AllArgsConstructor` pour l'injection des dépendances par constructeur (recommandée par Spring).

- **Opérations CRUD complètes :** Chaque entité dispose des méthodes `POST` (création), `PUT` (modification), `GET` (consultation de tous les enregistrements), `GET /getById/{id}` (consultation par identifiant), et `DELETE /delete/{id}` (suppression).
- **Respect de l'architecture 3-tiers :** Les contrôleurs n'accèdent jamais directement à la base de données : ils délèguent systématiquement tous les traitements à la couche **Service** (`IAgenceService`, `IClientService`, etc.).
- **Résolution des boucles infinies JSON :** Annotation de toutes les relations bidirectionnelles avec `@JsonIgnore` pour garantir des réponses JSON légères, rapides et sans exception de récursion cyclique.
- **Versionnement Git :** Code commité et poussé sur la branche `main` et `master` du dépôt GitHub public.

2. Cartographie Globale des Endpoints de l'API

Entité	Méthode	Endpoint (URI)	Rôle & Description
Agence		`/agence` / `/agence/getById/{id}`	Consulter toutes les agences / par ID
Agence		`/agence`	Ajouter une agence / Mettre à jour
Agence		`/agence/delete/{id}`	Supprimer une agence par ID
Client		`/client` / `/client/getById/{id}`	Consulter tous les clients / par ID
Client		`/client`	Ajouter un client / Mettre à jour
Client		`/client/delete/{id}`	Supprimer un client par ID
Contrat		`/contrat` / `/contrat/getById/{id}`	Consulter tous les contrats / par ID

Entité	Méthode	Endpoint (URI)	Rôle & Description
Contrat		`/contrat`	Ajouter un contrat / Mettre à jour
Contrat		`/contrat/delete/{id}`	Supprimer un contrat par ID
Employé		`/employe` / `/employe/getById/{id}`	Consulter tous les employés / par ID
Employé		`/employe`	Ajouter un employé / Mettre à jour
Employé		`/employe/delete/{id}`	Supprimer un employé par ID
Équipement		`/equipement` / `/equipement/getById/{id}`	Consulter tous les équipements / par ID
Équipement		`/equipement`	Ajouter un équipement / Mettre à jour
Équipement		`/equipement/delete/{id}`	Supprimer un équipement par ID
Maintenance		`/maintenance` / `/maintenance/getById/{id}`	Consulter toutes les maintenances / par ID
Maintenance		`/maintenance`	Planifier une maintenance / Mettre à jour
Maintenance		`/maintenance/delete/{id}`	Supprimer une maintenance par ID

Entité	Méthode	Endpoint (URI)	Rôle & Description
Paiement		`/paiement` / `/paiement/getById/{id}`	Consulter tous les paiements / par ID
Paiement		`/paiement`	Enregistrer un paiement / Mettre à jour
Paiement		`/paiement/delete/{id}`	Supprimer un paiement par ID
Réservation		`/reservation` / `/reservation/getById/{id}`	Consulter toutes les réservations / par ID
Réservation		`/reservation`	Créer une réservation / Mettre à jour
Réservation		`/reservation/delete/{id}`	Supprimer une réservation par ID
Véhicule		`/vehicule` / `/vehicule/getById/{id}`	Consulter tous les véhicules / par ID
Véhicule		`/vehicule`	Ajouter un véhicule / Mettre à jour
Véhicule		`/vehicule/delete/{id}`	Supprimer un véhicule par ID

3. Implémentation du Code et Résultats des Tests CRUD (Postman)

3.1. Contrôleur Agence (AgenceController)

Code Java :

```
package tn.esprit.autoloc.controllers; import lombok.AllArgsConstructor; import
org.springframework.web.bind.annotation.*; import tn.esprit.autoloc.entities.Agence; import
tn.esprit.autoloc.services.IAgenceService; import java.util.List; @RestController
@AllArgsConstructor @RequestMapping("/agence") public class AgenceController { private
final IAgenceService agenceService; @PostMapping public Agence ajouterAgence(@RequestBody
Agence agence) { return agenceService.ajouterAgence(agence); } @PutMapping public Agence
modifierAgence(@RequestBody Agence agence) { return agenceService.modifierAgence(agence); }
@GetMapping public List<Agence> listerAgences() { return agenceService.consulterAgences();
} @GetMapping("/getById/{idAgence}") public Agence getAgenceById(@PathVariable Long
idAgence) { return agenceService.consulterAgenceById(idAgence); }
@DeleteMapping("/delete/{idAgence}") public void supprimerAgence(@PathVariable Long
idAgence) { agenceService.supprimerAgence(idAgence); } }
```

Résultats des Tests CRUD avec Postman :

```
POST http://localhost:8083/agence Status: 200 OK • Time: 15 ms
```

Request Body (JSON) :

```
{ "nom": "AutoLoc Nabeul", "ville": "Nabeul", "adresse": "Avenue Habib Thameur", "telephone":
"+216 72 333 444" }
```

Response Body (JSON) :

```
{ "idAgence": 3, "nom": "AutoLoc Nabeul", "ville": "Nabeul", "adresse": "Avenue Habib Thameur",
"telephone": "+216 72 333 444" }
```

[Insérer la capture d'écran Postman : AgenceController • GET / POST]

3.2. Contrôleur Client (ClientController)

Code Java :

```
package tn.esprit.autoloc.controllers; import lombok.AllArgsConstructor; import
org.springframework.web.bind.annotation.*; import tn.esprit.autoloc.entities.Client; import
tn.esprit.autoloc.services.IClientService; import java.util.List; @RestController
@AllArgsConstructor @RequestMapping("/client") public class ClientController { private
final IClientService clientService; @PostMapping public Client ajouterClient(@RequestBody
Client client) { return clientService.ajouterClient(client); } @PutMapping public Client
modifierClient(@RequestBody Client client) { return clientService.modifierClient(client); }
@GetMapping public List<Client> listerClients() { return clientService.consulterClients();
} @GetMapping("/getById/{idClient}") public Client getClientById(@PathVariable Long
```

```
idClient) { return clientService.consulterClientById(idClient); }
@DeleteMapping("/delete/{idClient}") public void supprimerClient(@PathVariable Long
idClient) { clientService.supprimerClient(idClient); } }
```

Résultats des Tests CRUD avec Postman :

```
POST http://localhost:8083/client Status: 200 OK • Time: 12 ms
```

Request Body (JSON) :

```
{ "nom": "Trabelsi", "prenom": "Bilel", "email": "bilel.trabelsi@esprit.tn", "telephone": "+216
22 555 666", "numPermis": "TN-445566", "dateInscription": "2026-10-23" }
```

Response Body (JSON) :

```
{ "idClient": 3, "nom": "Trabelsi", "prenom": "Bilel", "email": "bilel.trabelsi@esprit.tn",
"telephone": "+216 22 555 666", "numPermis": "TN-445566", "dateInscription": "2026-10-23" }
```

[Insérer la capture d'écran Postman : ClientController • GET / POST]

3.3. Contrôleur Contrat (ContratController)

Code Java :

```
package tn.esprit.autoloc.controllers; import lombok.AllArgsConstructor; import
org.springframework.web.bind.annotation.*; import tn.esprit.autoloc.entities.Contrat;
import tn.esprit.autoloc.services.IContratService; import java.util.List; @RestController
@AllArgsConstructor @RequestMapping("/contrat") public class ContratController { private
final IContratService contratService; @PostMapping public Contrat
ajouterContrat(@RequestBody Contrat contrat) { return
contratService.ajouterContrat(contrat); } @PutMapping public Contrat
modifierContrat(@RequestBody Contrat contrat) { return
contratService.modifierContrat(contrat); } @GetMapping public List<Contrat>
listerContrats() { return contratService.consulterContrats(); }
@GetMapping("/getById/{idContrat}") public Contrat getContratById(@PathVariable Long
idContrat) { return contratService.consulterContratById(idContrat); }
@DeleteMapping("/delete/{idContrat}") public void supprimerContrat(@PathVariable Long
idContrat) { contratService.supprimerContrat(idContrat); } }
```

Résultats des Tests CRUD avec Postman :

```
POST http://localhost:8083/contrat Status: 200 OK • Time: 14 ms
```

Request Body (JSON) :

```
{ "dateSignature": "2026-10-23", "montantTotal": 520.00, "valide": true }
```

Response Body (JSON) :

```
{ "idContrat": 2, "dateSignature": "2026-10-23", "montantTotal": 520.00, "valide": true }
```

[Insérer la capture d'écran Postman : ContratController • GET / POST]

3.4. Contrôleur Employé (EmployeController)

Code Java :

```
package tn.esprit.autoloc.controllers; import lombok.AllArgsConstructor; import
org.springframework.web.bind.annotation.*; import tn.esprit.autoloc.entities.Employe;
import tn.esprit.autoloc.services.IEmployeService; import java.util.List; @RestController
@AllArgsConstructor @RequestMapping("/employe") public class EmployeController { private
final IEmployeService employeService; @PostMapping public Employe
ajouterEmploye(@RequestBody Employe employe) { return
employeService.ajouterEmploye(employe); } @PutMapping public Employe
modifierEmploye(@RequestBody Employe employe) { return
employeService.modifierEmploye(employe); } @GetMapping public List<Employe>
listerEmployes() { return employeService.consulterEmployes(); }
@GetMapping("/getById/{idEmploye}") public Employe getEmployeById(@PathVariable Long
idEmploye) { return employeService.consulterEmployeById(idEmploye); }
@DeleteMapping("/delete/{idEmploye}") public void supprimerEmploye(@PathVariable Long
idEmploye) { employeService.supprimerEmploye(idEmploye); } }
```

Résultats des Tests CRUD avec Postman :

```
POST http://localhost:8083/employe Status: 200 OK • Time: 16 ms
```

Request Body (JSON) :

```
{ "nom": "Khmiri", "prenom": "Omar", "role": "AGENT", "agence": { "idAgence": 1 } }
```

Response Body (JSON) :

```
{ "idEmploye": 4, "nom": "Khmiri", "prenom": "Omar", "role": "AGENT", "agence": { "idAgence": 1,
"nom": "AutoLoc Tunis Centre", "ville": "Tunis", "adresse": "Avenue Habib Bourguiba, Tunis",
"telephone": "+216 71 123 456" } }
```

[Insérer la capture d'écran Postman : EmployeController • GET / POST]

3.5. Contrôleur Équipement (EquipementController)

Code Java :

```
package tn.esprit.autoloc.controllers; import lombok.AllArgsConstructor; import
org.springframework.web.bind.annotation.*; import tn.esprit.autoloc.entities.Equipement;
import tn.esprit.autoloc.services.IEquipementService; import java.util.List;
@RestController @AllArgsConstructor @RequestMapping("/equipement") public class
EquipementController { private final IEquipementService equipementService; @PostMapping
public Equipement ajouterEquipement(@RequestBody Equipement equipement) { return
```

```
equipementService.ajouterEquipement(equipement); } @PutMapping public Equipement
modifierEquipement(@RequestBody Equipement equipement) { return
equipementService.modifierEquipement(equipement); } @GetMapping public List<Equipement>
listerEquipements() { return equipementService.consulterEquipements(); }
@GetMapping("/getById/{idEquipement}") public Equipement getEquipementById(@PathVariable
Long idEquipement) { return equipementService.consulterEquipementById(idEquipement); }
@DeleteMapping("/delete/{idEquipement}") public void supprimerEquipement(@PathVariable Long
idEquipement) { equipementService.supprimerEquipement(idEquipement); } }
```

Résultats des Tests CRUD avec Postman :

```
POST http://localhost:8083/equipement Status: 200 OK • Time: 10 ms
```

Request Body (JSON) :

```
{ "libelle": "Toit panoramique ouvrant" }
```

Response Body (JSON) :

```
{ "idEquipement": 5, "libelle": "Toit panoramique ouvrant" }
```

[Insérer la capture d'écran Postman : EquipementController • GET / POST]

3.6. Contrôleur Maintenance (MaintenanceController)

Code Java :

```
package tn.esprit.autoloc.controllers; import lombok.AllArgsConstructor; import
org.springframework.web.bind.annotation.*; import tn.esprit.autoloc.entities.Maintenance;
import tn.esprit.autoloc.services.IMaintenanceService; import java.util.List;
@RestController @AllArgsConstructor @RequestMapping("/maintenance") public class
MaintenanceController { private final IMaintenanceService maintenanceService; @PostMapping
public Maintenance ajouterMaintenance(@RequestBody Maintenance maintenance) { return
maintenanceService.ajouterMaintenance(maintenance); } @PutMapping public Maintenance
modifierMaintenance(@RequestBody Maintenance maintenance) { return
maintenanceService.modifierMaintenance(maintenance); } @GetMapping public List<Maintenance>
listerMaintenances() { return maintenanceService.consulterMaintenances(); }
@GetMapping("/getById/{idMaintenance}") public Maintenance getMaintenanceById(@PathVariable
Long idMaintenance) { return maintenanceService.consulterMaintenanceById(idMaintenance); }
@DeleteMapping("/delete/{idMaintenance}") public void supprimerMaintenance(@PathVariable
Long idMaintenance) { maintenanceService.supprimerMaintenance(idMaintenance); } }
```

Résultats des Tests CRUD avec Postman :

```
POST http://localhost:8083/maintenance Status: 200 OK • Time: 18 ms
```

Request Body (JSON) :

```
{ "dateDebut": "2026-10-24", "dateFin": "2026-10-26", "description": "Revision periodique et
filtres", "vehicule": { "idVehicule": 1 } }
```

Response Body (JSON) :


```
{ "idMaintenance": 2, "dateDebut": "2026-10-24", "dateFin": "2026-10-26", "description":
"Revision periodique et filtres", "vehicule": { "idVehicule": 1, "immatriculation": "230-TN-
1024", "marque": "Renault", "modele": "Clio 5" } }
```


[Insérer la capture d'écran Postman : MaintenanceController • GET / POST]

3.7. Contrôleur Paiement (PaiementController)

Code Java :

```
package tn.esprit.autoloc.controllers; import lombok.AllArgsConstructor; import
org.springframework.web.bind.annotation.*; import tn.esprit.autoloc.entities.Paiement;
import tn.esprit.autoloc.services.IPaiementService; import java.util.List; @RestController
@AllArgsConstructor @RequestMapping("/paiement") public class PaiementController { private
final IPaiementService paiementService; @PostMapping public Paiement
ajouterPaiement(@RequestBody Paiement paiement) { return
paiementService.ajouterPaiement(paiement); } @PutMapping public Paiement
modifierPaiement(@RequestBody Paiement paiement) { return
paiementService.modifierPaiement(paiement); } @GetMapping public List<Paiement>
listerPaiements() { return paiementService.consulterPaiements(); }
@GetMapping("/getById/{idPaiement}") public Paiement getPaiementById(@PathVariable Long
idPaiement) { return paiementService.consulterPaiementById(idPaiement); }
@DeleteMapping("/delete/{idPaiement}") public void supprimerPaiement(@PathVariable Long
idPaiement) { paiementService.supprimerPaiement(idPaiement); } }
```

Résultats des Tests CRUD avec Postman :

```
POST http://localhost:8083/paiement Status: 200 OK • Time: 19 ms
```

Request Body (JSON) :


```
{ "datePaiement": "2026-10-23", "montant": 450.00, "modePaiement": "CARTE_BANCAIRE", "contrat": {
"idContrat": 1 } }
```


Response Body (JSON) :


```
{ "idPaiement": 2, "datePaiement": "2026-10-23", "montant": 450.00, "modePaiement":
"CARTE_BANCAIRE", "contrat": { "idContrat": 1, "dateSignature": "2026-10-05", "montantTotal":
880.00, "valide": true } }
```


[Insérer la capture d'écran Postman : PaiementController • GET / POST]

3.8. Contrôleur Réservation (ReservationController)

Code Java :

```
package tn.esprit.autoloc.controllers; import lombok.AllArgsConstructor; import
org.springframework.web.bind.annotation.*; import tn.esprit.autoloc.entities.Reservation;
import tn.esprit.autoloc.services.IReservationService; import java.util.List;
@RestController @AllArgsConstructor @RequestMapping("/reservation") public class
ReservationController { private final IReservationService reservationService; @PostMapping
public Reservation ajouterReservation(@RequestBody Reservation reservation) { return
reservationService.ajouterReservation(reservation); } @PutMapping public Reservation
modifierReservation(@RequestBody Reservation reservation) { return
reservationService.modifierReservation(reservation); } @GetMapping public List<Reservation>
listerReservations() { return reservationService.consulterReservations(); }
@GetMapping("/getById/{idReservation}") public Reservation getReservationById(@PathVariable
Long idReservation) { return reservationService.consulterReservationById(idReservation); }
@DeleteMapping("/delete/{idReservation}") public void supprimerReservation(@PathVariable
Long idReservation) { reservationService.supprimerReservation(idReservation); } }
```

Résultats des Tests CRUD avec Postman :

```
POST http://localhost:8083/reservation Status: 200 OK • Time: 21 ms
```

Request Body (JSON) :

```
{ "dateDebut": "2026-11-10", "dateFin": "2026-11-15", "statut": "EN_ATTENTE", "client": {
"idClient": 1 }, "vehicule": { "idVehicule": 1 } }
```

Response Body (JSON) :

```
{ "idReservation": 2, "dateDebut": "2026-11-10", "dateFin": "2026-11-15", "statut": "EN_ATTENTE",
"client": { "idClient": 1, "nom": "Ayari", "prenom": "Youssef" }, "vehicule": { "idVehicule": 1,
"immatriculation": "230-TN-1024", "marque": "Renault", "modele": "Clio 5" } }
```

[Insérer la capture d'écran Postman : ReservationController • GET / POST]

3.9. Contrôleur Véhicule (VehiculeController)

Code Java :

```
package tn.esprit.autoloc.controllers; import lombok.AllArgsConstructor; import
org.springframework.web.bind.annotation.*; import tn.esprit.autoloc.entities.Vehicule;
import tn.esprit.autoloc.services.IVehiculeService; import java.util.List; @RestController
@AllArgsConstructor @RequestMapping("/vehicule") public class VehiculeController { private
final IVehiculeService vehiculeService; @PostMapping public Vehicule
ajouterVehicule(@RequestBody Vehicule vehicule) { return
vehiculeService.ajouterVehicule(vehicule); } @PutMapping public Vehicule
modifierVehicule(@RequestBody Vehicule vehicule) { return
vehiculeService.modifierVehicule(vehicule); } @GetMapping public List<Vehicule>
listerVehicules() { return vehiculeService.consulterVehicules(); }
@GetMapping("/getById/{idVehicule}") public Vehicule getVehiculeById(@PathVariable Long
idVehicule) { return vehiculeService.consulterVehiculeById(idVehicule); }
@DeleteMapping("/delete/{idVehicule}") public void supprimerVehicule(@PathVariable Long
idVehicule) { vehiculeService.supprimerVehicule(idVehicule); } }
```

Résultats des Tests CRUD avec Postman :

```
POST http://localhost:8083/vehicule Status: 200 OK • Time: 20 ms
```

Request Body (JSON) :

```
{ "immatriculation": "255-TN-7788", "marque": "Volkswagen", "modele": "Golf 8", "categorie":
"BERLINE", "tarifJournalier": 160.00, "statut": "DISPONIBLE", "agence": { "idAgence": 1 } }
```

Response Body (JSON) :

```
{ "idVehicule": 4, "immatriculation": "255-TN-7788", "marque": "Volkswagen", "modele": "Golf 8",
"categorie": "BERLINE", "tarifJournalier": 160.00, "statut": "DISPONIBLE", "agence": {
"idAgence": 1, "nom": "AutoLoc Tunis Centre", "ville": "Tunis" } }
```

[Insérer la capture d'écran Postman : VehiculeController • GET / POST]

4. Conclusion & Références

La couche présentation du projet **AutoLoc** est désormais achevée, robuste et entièrement documentée :

- **Lien du Dépôt GitHub :** https://github.com/salim-hechmi/AutoLoc
- **Collection Postman :** Incluse dans le dépôt sous le nom `AutoLoc_Postman_Collection.json`.
- **Documentation Swagger UI :** Accessible en local sur http://localhost:8083/swagger-ui/index.html.